

Section 2 — System Design

1. Problem Statement

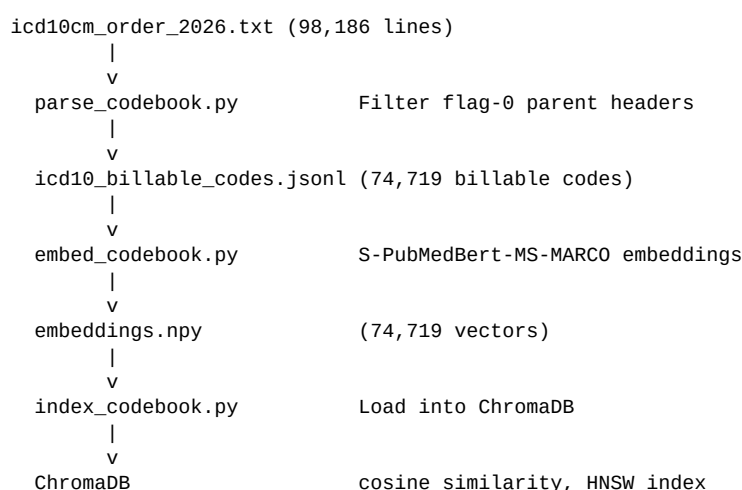
Having 54 clinical notes as queries is challenging to identify the right medical codes. Medical cases are written in informal clinical language with multiple distinct diagnoses within a case. There is a vernacular gap: clinicians write e.g. "herniated disc" whereas the codebook states "intervertebral disc displacement". The codebook contains many similar codes with minor differences between each other (encounter type, severity).

The search space consists of 98,186 lines in the raw ICD-10-CM codebook, of which 74,719 are billable codes (flag-1) after filtering parent category headers (flag-0). The system must accurately assign codes from this large codebook to each clinical note.

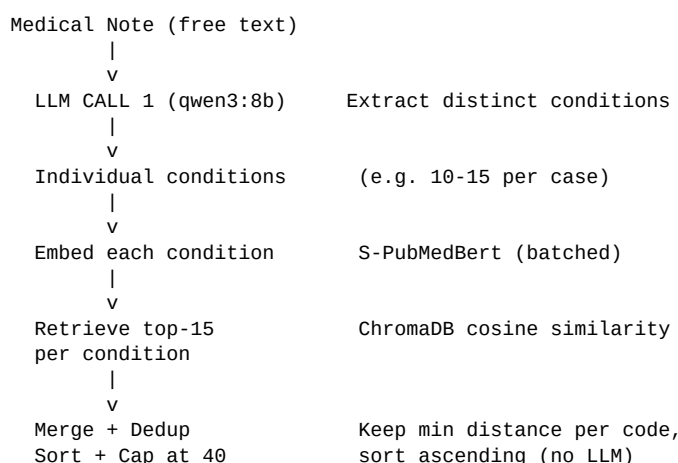
2. Architecture Overview

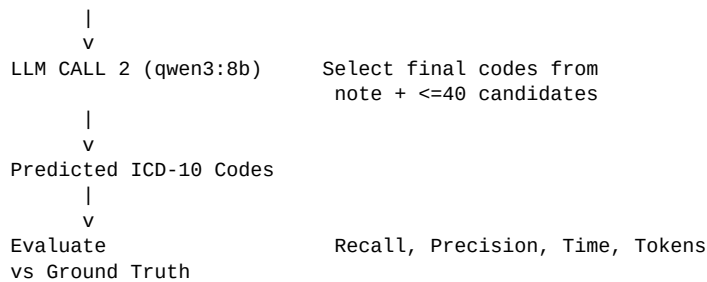
The pipeline has two phases: an offline indexing phase (run once) and an online inference phase (per case).

Offline Pipeline (run once)



Online Pipeline (per case)





3. Key Design Decisions

Embedding Model — S-PubMedBert-MS-MARCO

PubMedBERT is pretrained on biomedical terminology, MS-MARCO fine-tuned for better retrieval results. However, it is trained on medical journal texts and not ICD billing labels — the vernacular gap still exists.

LLM — qwen3:8b via Ollama

Local and offline, open-sourced. Hardware limitations considered (MacBook M4, 16GB unified memory), hence decided on 8B parameters only. The model requires approximately 5GB of RAM for weights, leaving room for the embedding model (~400MB), ChromaDB (~1GB) and macOS (~4-5GB).

Query Decomposition — The Critical Insight

Initially tried to embed the entire medical note as a single vector. However, results showed approximately zero recall. Two possible issues: 'k' retrieval was too small if the correct code is far down in the ranks, or the single-vector embedding was too weak — not grasping the correct medical code in relation to the medical case at all (regardless of how many 'k' I retrieve).

Developed a retrieval diagnostic script to test if ground truth codes are even in the candidates at all. Set retrieval to 100 per medical case. For every correct code retrieved, I recorded its rank position and cosine distance. Result: only 28/158 ground truth codes were retrieved in the top 100.

Found the root cause to be embedding-related. The medical note was too comprehensive — containing multiple distinct diagnoses which diluted the embedding space, thus retrieving the wrong codes from the database. This is the same dilution problem I identified in my previous project (GroundedSG, see Section 1).

Redesigned retrieval using query decomposition. When the medical case is queried, I use qwen3 to split the medical note into distinct conditions. Each condition is then embedded individually with S-PubMedBert-MS-MARCO and retrieves 15 codes per distinct condition.

There were many identical codes fetched within the same medical case even with different distinct conditions. Hence, added a deterministic de-duplicate function to reduce the total to ≤40 codes retrieved. Finally, call qwen3 LLM to select codes to answer the medical case.

Temperature 0.0

With 0 temperature, the model is deterministic which allows the same retrieved codes each time we run the program. This allows Version A and B comparisons to be fair.

Cap Final LLM Output to 100 Tokens

100 tokens is enough to select from a list of final codes. This acts as a safety brake — during trials the LLM fell into an infinite loop, repeating code sequences indefinitely, hanging the system.

4. Version Comparison

Version A — Retrieval Only

Set k=1: each medical case has several distinct conditions so approximately 8-15 total codes retrieved. Since Version A has no filtering afterwards, the retrieved chunks are presented as the answer itself. If k were any larger, the number of retrieved chunks would tank precision. The final number of output codes is uncapped.

Version B — Retrieval + LLM Selection

Set k=15: retrieving 15 codes per distinct condition, because we have an additional filtering layer available, we can afford to grab as many relevant answers as possible (with the cost of tokens and time). Total retrieval code pool is approximately 130. Further filtering with a deterministic rule to de-duplicate any repeated code and sort them by distance (closest distance = smallest number). As the codes are sorted by distance (regardless of distinct condition type), I select the top 40 codes and pass it to qwen3 to filter down to the final answer.

Previously, I included all ~130 retrieved codes to the LLM for a final answer. However the LLM output long essays and category explanations instead of a code list. Capping at 40 candidates resolved this.

Metric	Version A (retrieval only)	Version B (retrieval + LLM)
Avg recall	0.308	0.609
Avg precision	0.048	0.237
Avg time/case	6.06s	15.11s

The version with augmentation LLM nearly doubles the recall (0.31 → 0.61), and quintuples the precision (0.05 → 0.24) at the cost of 2.5x slower inference. Hence I decided to go with the LLM.

5. Final Results — All 54 Cases

Metric	Value
Average recall	0.609
Average precision	0.237
Retrieval recall	0.671 (106/158 ground truth codes reached pool)
LLM retention	91% of recoverable codes selected
Avg time/case	15.11s
Total tokens	70,751 (67,443 prompt + 3,308 completion)

Evaluating the pipeline as a whole, I determine retrieval still needs more work as 33% of ground truth codes never reach the retrieval pool. The LLM selects the right codes from the pool 91% of the time, hence including a final-stage LLM augmentation is necessary.

6. Debugging Journey

qwen3 reasoning mode `</think>` tags weren't working as expected even after setting 'think: false' on its API flag. Wrote a small function to remove any `"/think"` texts from answers.

During trials, the LLM fell into an infinite loop of predicting the next most probable token repeatedly with temperature 0.0 (greedy decoding). Without a hard cap on output tokens, the model repeated code sequences indefinitely. Added `num_predict: 100` as a safety brake.

When passing all ~130 merged candidates to the LLM, it output markdown essays and category explanations instead of code lists. Capping candidates at 40 resolved this.

Early 10-case tests showed precision of 0.10; full 54-case run showed 0.237. Lesson: small samples are noisy — always validate on the full dataset.

7. Limitations

- Retrieval ceiling was 67.1%. 52 of 158 ground truth codes did not reach the candidate pool.
- Embedding model trained on PubMed prose, not ICD billing terminology.
- No hybrid search within pipeline.
- Pipeline predicts approximately 5 codes per case, but only ~1 is in the ground truth. Precision is 0.237.
- While extracting distinct diagnoses/conditions from each medical case, it is left to the LLM to decide. If any conditions are merged together and depicted as a single distinct condition, it is left unchecked and its embeddings become diluted.

8. Future Improvements

Codebook enrichment: The codebook database only consists of bare official code + short clinical description. It does not have synonyms, inclusion terms nor alternate wordings. Instead we could pull official files from CMS/NCHS and extract the extra terms per medical code to add to our codebook database before embedding it. This closes the vernacular gap and improves retrieval significantly.

Hybrid retrieval: Combine semantic (embedding) and lexical search (BM25) to catch exact term matching that embeddings miss.

Few-shot prompting: Show 2–3 example queries and a list of selected retrieved pool of medical codes. Show the correct ideal medical code as the answer with reasoning. This helps the model copy the selection pattern and reason through ambiguous cases right there in the context.

Structured JSON output: The final LLM layer outputs "/think" reasoning tags or long essays when we just want medical codes as answers. Having structured JSON output formats prevents this.

Self-consistency: The final LLM layer selects from a pool of codes once and we trust its output. The output may be inconsistent at times. Instead we could ask the same LLM to select a few times with varying temperature levels, and based on the selected medical codes each run picked, we keep only the repeated ones across a majority of runs. Tradeoff is more LLM runs, tokens and time.

Hallucination guardrail: Right now the LLM is instructed to select from a pool of retrieved medical codes. However it may hallucinate or invent its own code that was never in the list. Check the final answer against our retrieved medical codes as a guardrail and drop hallucinated ones.

Encounter-type consistency: The medical case query might not indicate information such as "initial visit" or "follow-up". As such, encounter types in the codebook — A (initial encounter), D (subsequent encounter), S (sequela) — e.g. T18128A and T18128D might both be present in the final answers. A guardrail could detect contradictory encounter-type suffixes for the same condition and flag or discard the conflict.